

CSAPP 3장: 프로그램의 기계 수준 표현

심화편 (3.1 ~ 3.3) | 입문편을 먼저 읽고 오기

0. 먼저 큰 그림부터

이 챕터가 말하려는 것 (한 줄 요약)

우리가 짠 C 코드가 컴퓨터가 실제로 실행하는 **기계어(machine code)**로 어떻게 변환되는지, 그리고 그 중간 단계인 **어셈블리어(assembly)**를 읽는 법을 배운다.

비유: C 코드 = 한국어로 쓴 요리 레시피, 어셈블리 = 아주 단순한 영어 명령어들("계란 깨라", "소금 넣어라"), 기계어 = 바코드(컴퓨터만 읽을 수 있는 숫자 나열). 컴파일러가 한국어 레시피를 바코드로 번역해주는 번역기 역할을 한다.

3.1~3.3에서 배울 것:

- 3.1** — x86 프로세서의 역사 (가볍게 훑기)
- 3.2** — C 코드가 기계어가 되는 과정 (핵심!)
- 3.3** — 데이터 크기와 포맷 (핵심!)

3.1 역사적 배경 (A Historical Perspective)

깊게 외울 필요 없다. **흐름만 알면 된다:**

핵심만

- Intel이 1978년에 **8086** (16비트)을 만들었고, 그게 계속 진화해서 지금의 **x86-64** (64비트)가 되었다
- x86이라는 이름은 8086 → 80286 → i386 → i486... 이렇게 "86"이 계속 붙어서 생긴 별명이다
- 16비트 → 32비트(i386, 1985) → 64비트(x86-64, 2004)** 이 흐름이 중요하다
- 32비트는 메모리를 최대 4GB까지만 쓸 수 있었다. 64비트는 이론상 16EB(엑사바이트)까지 가능
- 새 프로세서는 항상 **하위 호환(backward compatible)** — 옛날 코드도 돌아간다

비유: iPhone 1 → iPhone 16처럼 계속 업그레이드됐는데, 옛날 앱도 여전히 돌아가는 것과 비슷하다. 그래서 x86-64에는 옛날 잔재가 좀 남아있다.

무어의 법칙 (Moore's Law)

트랜지스터 수가 약 **18개월마다 2배**로 늘어난다는 관찰. 1978년 8086은 트랜지스터 29,000개였는데, 2013년 Haswell은 14억 개. 이게 컴퓨터가 계속 빨라진 핵심 원동력이다.

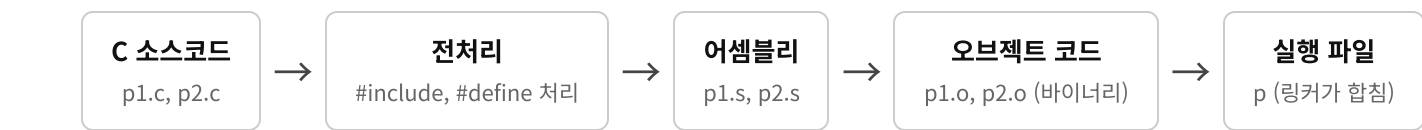
3.2 프로그램 인코딩 (Program Encodings) ★핵심

3.2.1 C 코드 → 실행 파일이 되는 과정

C 파일 두 개(p1.c , p2.c)를 컴파일한다고 하자:

```
linux> gcc -0g -o p p1.c p2.c
```

이 한 줄 명령어 뒤에서 **4단계**가 일어난다:



각 단계 설명

1. 전처리기(Preprocessor):

#include 로 불러온 파일을 붙여넣고, #define 매크로를 치환한다

2. 컴파일러(Compiler):

C 코드를 어셈블리 코드(.s 파일)로 변환한다. 여기가 가장 복잡한 단계

3. 어셈블러(Assembler):

어셈블리 코드를 오브젝트 코드(.o 파일, 바이너리)로 변환한다

4. 링커(Linker):

여러 .o 파일과 라이브러리(printf 등)를 합쳐서 최종 실행 파일을 만든다

gcc 옵션 정리 (시험에 나올 수 있음)

명령어	하는 일	결과물
gcc -0g -S mstore.c	어셈블리까지만 생성	mstore.s (텍스트)
gcc -0g -c mstore.c	오브젝트 코드까지 생성	mstore.o (바이너리)
gcc -0g -o prog main.c mstore.c	실행 파일까지 전부	prog (실행 가능)
objdump -d mstore.o	바이너리 → 어셈블리 역변환	디스어셈블리 출력

-0g 는 "최적화를 최소한으로"라는 뜻이다. 디버깅/학습용. 실무에서는 -01 , -02 를 쓴다.

3.2.2 실제 예제로 보기

이 C 함수를 보자:

```

long mult2(long, long);

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}

```

이걸 gcc -Og -S 로 컴파일하면 이런 어셈블리가 나온다:

```

multstore:
    pushq    %rbx          # rbx 레지스터를 스택에 백업
    movq     %rdx, %rbx     # dest(세 번째 인자)를 rbx에 복사
    call     mult2          # mult2(x, y) 호출
    movq     %rax, (%rbx)    # 결과를 *dest에 저장
    popq     %rbx          # rbx 복원
    ret                     # 리턴

```

여기서 읽는 법

- %rax , %rbx , %rdx — 이런 건 **레지스터** 이름이다 (CPU 안의 작은 저장 공간)
- pushq , movq , call , ret — **명령어(instruction)**. 뒤에 붙는 q 는 "quad word(8바이트)"라는 뜻
- (%rbx) — 괄호는 "그 주소에 있는 메모리"라는 뜻. C의 *ptr 과 같다

비유: C 코드가 "x와 y를 곱해서 dest가 가리키는 곳에 저장해"라고 했다면, 어셈블리는 이것 아주 작은 단계로 쪼갠 것이다. "먼저 rbx를 백업해놔 → dest 주소를 rbx에 넣어 → mult2 함수를 불러 → 결과를 rbx가 가리키는 메모리에 넣어 → rbx 복원해 → 돌아가"

3.2.3 기계 수준 코드의 핵심 개념

CPU가 보는 세계 (C 프로그래머에게는 숨겨진 것들)

1. **프로그램 카운터 (PC, %rip):** "다음에 실행할 명령어의 주소"를 가리킨다
2. **레지스터 파일:** 16개의 64비트 레지스터. 변수/주소/임시값을 저장하는 초고속 저장 공간
3. **조건 코드 레지스터:** 마지막 연산 결과의 상태 정보 (양수? 음수? 0? 오버플로우?). if문 구현에 사용
4. **메모리:** 기계어 관점에서는 그냥 "거대한 바이트 배열"이다. C에서의 int, char, struct 구분이 없다

비유: C에서는 "이건 int고, 저건 char*야"라고 타입을 구분하지만, 기계어 수준에서는 전부 그냥 "숫자 덩어리"다. 타입 같은 건 컴파일러가 알아서 처리해주고, CPU는 모른다.

디스어셈블리 (Disassembly)

바이너리 파일(.o)을 다시 어셈블리로 변환해서 보는 것:

```
linux> objdump -d mstore.o

0000000000000000 <mstore>:
 0: 53          push    %rbx
 1: 48 89 d3    mov     %rdx,%rbx
 4: e8 00 00 00 00 callq   9 <mstore+0x9>
 9: 48 89 03    mov     %rax,(%rbx)
c: 5b          pop     %rbx
d: c3          retq
```

읽는 법

왼쪽 숫자 = 메모리 주소(offset), 가운데 = 기계어(바이트), 오른쪽 = 어셈블리

예: 53 한 바이트가 push %rbx 라는 명령어다. 48 89 d3 세 바이트가 mov %rdx,%rbx 라는 명령어다.

x86-64 명령어는 1~15바이트로 길이가 제각각이다. 자주 쓰는 명령어일수록 짧다.

ATT vs Intel 형식

이 수업에서는 **ATT 형식**(gcc 기본)을 쓴다. Intel 형식과 다른 점:

	ATT (우리가 쓰는 것)	Intel
레지스터	%rbx	rbx
크기 접미사	movq	mov
메모리 참조	(%rbx)	QWORD PTR [rbx]
피연산자 순서	src, dst	dst, src (반대!)

3.3 데이터 포맷 (Data Formats) ★핵심

x86-64에서 각 C 데이터 타입이 몇 바이트이고, 어셈블리에서 어떤 접미사를 쓰는지:

C 타입	Intel 용어	어셈블리 접미사	크기
char	Byte	b	1바이트 (8비트)
short	Word	w	2바이트 (16비트)
int	Double word	l	4바이트 (32비트)
long	Quad word	q	8바이트 (64비트)

C 타입	Intel 용어	어셈블리 접미사	크기
char * (포인터)	Quad word	q	8바이트 (64비트)
float	Single precision	s	4바이트
double	Double precision	l	8바이트

반드시 외워야 할 것

- **"Word"가 16비트**라는 것 — Intel이 16비트 시절에 시작해서 "word = 16비트"로 정했다. 보통 다른 아키텍처에서 word가 32비트인 것과 다르다!
- **접미사 b, w, l, q** — 명령어 뒤에 붙어서 "몇 바이트짜리 데이터를 다루는지"를 알려준다
 - movb = 1바이트 이동, movw = 2바이트, movl = 4바이트, movq = 8바이트
- **포인터는 항상 8바이트** (64비트 머신이니까)
- `l` 이 int(4바이트)와 double(8바이트) 둘 다에 쓰인다. 하지만 정수 명령어와 부동소수점 명령어가 완전히 별개라서 헷갈릴 일은 없다

왜 "double word"가 32비트인가? Intel이 처음에 16비트(= 1 word)로 시작했으니까, 32비트는 word의 2배 = double word, 64비트는 word의 4배 = quad word. 이름이 역사적 이유로 이렇게 된 것이다.

접미사 암기법

b = byte	(1바이트)	→ char	
w = word	(2바이트)	→ short	
l = long	(4바이트)	→ int	← "long word"의 약자
q = quad	(8바이트)	→ long, 포인터	

퀴즈 대비: 핵심 질문

이 질문들에 대답할 수 있으면 3.1~3.3은 이해한 것이다:

1. **gcc -S와 gcc -c의 차이는?**
→ -S는 어셈블리(.s)까지만, -c는 오브젝트 코드(.o)까지
2. **C 코드 → 실행 파일까지 4단계를 순서대로 말해라**
→ 전처리 → 컴파일(어셈블리 생성) → 어셈블(오브젝트 코드) → 링크(실행 파일)
3. **objdump -d는 뭘 하는 건가?**
→ 바이너리(기계어)를 어셈블리로 역변환(디스어셈블리)해서 보여준다
4. **x86-64에서 "word"는 몇 비트인가?**
→ 16비트. (보통의 "word = 32비트"와 다르다!)

5. movq에서 q는 무슨 뜻인가?

→ quad word = 8바이트(64비트) 데이터를 다루겠다는 의미

6. int는 어셈블리에서 어떤 접미사를 쓰는가?

→ l (double word, 4바이트)

7. 포인터(char *)는 x86-64에서 몇 바이트인가?

→ 8바이트 (64비트 머신이니깐)

8. 프로그램 카운터(%rip)의 역할은?

→ 다음에 실행할 명령어의 메모리 주소를 저장한다

9. ATT 형식에서 피연산자 순서는?

→ src(출발), dst(도착) 순서. Intel은 반대다

10. 기계어 수준에서 int와 char *를 구분할 수 있는가?

→ 없다. 전부 그냥 바이트 덩어리. 타입은 컴파일러만 안다

부록: 이 챕터를 읽기 위한 C 최소 지식

C를 모른다고 했으니, 이 챕터에 나오는 C 문법만 빠르게 정리:

데이터 타입

```
char    c = 'A';    // 1바이트 문자
short   s = 100;    // 2바이트 정수
int      i = 42;     // 4바이트 정수
long     l = 100000; // 8바이트 정수 (x86-64에서)
float    f = 3.14;   // 4바이트 실수
double  d = 3.14159; // 8바이트 실수
```

포인터 (제일 중요)

```
int a = 10;
int *p = &a;    // p는 a의 "메모리 주소"를 저장
                // &는 "주소 가져오기"

*p = 20;        // *는 "그 주소에 가서 값을 읽거나 쓰기"
                // 이제 a는 20이 됨

// 어셈블리에서:
// (%rbx) ← 이제 C의 *p와 같은 것
// rbx에 저장된 주소로 가서 그 메모리를 읽겠다는 뜻
```

어셈블리 읽을 때 이것만 기억하면 된다

- %rax → 레지스터 이름 (CPU 안의 변수라고 생각)
- \$42 → 즉시값 (상수 42)
- (%rax) → rax에 담긴 주소의 메모리 = C의 *ptr

- `movq A, B` → A의 값을 B로 복사 ($A \rightarrow B$)

함수

```
// C에서의 함수
long mult2(long a, long b) {
    return a * b;
}

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y); // mult2 호출
    *dest = t;           // 결과를 dest가 가리키는 곳에 저장
}

// x86-64에서 함수 인자는 레지스터로 전달된다:
// 1번째 인자 → %rdi
// 2번째 인자 → %rsi
// 3번째 인자 → %rdx
// 리턴값      → %rax
```